MyCodeSchool

Testare cu JUnit și Mockito — exerciții

Alex · 10 exerciții: ghidate, practică, provocări.

Proiectul e magazinul online din `src/`. Testele le scrii în `test/`, în același pachet ca clasa testată. Modelul e `test/app/system/cart/CartTest.java`: îl citești întâi, apoi rulezi `mvn test` și vezi `Tests run: 1, Failures: 0`.

Fiecare test: `// Arrange`, `// Act`, `// Assert`, fără bucle, numele spune ce se întâmplă și când. După fiecare exercițiu, `mvn test` e verde.

Exercițiul 1 · Primul test, fără mock

Clasa `ProductTest` în `test/app/products/model/`. `Product` are un constructor care citește o linie din `Products.txt`: `id,nume,pret,greutate,categorie,stoc`.

1. `constructorReadsAllFieldsFromLine`: din `"1,Laptop,3000,2000,IT,5"` ies id `1`, nume `Laptop`, preț `3000`, greutate `2000`, categorie `IT`, stoc `5`. Câte un `assertEquals` pe fiecare câmp.
2. `toStringReturnsTheSameLine`: `toString()` pe produsul de mai sus dă exact linia din care a fost creat.
3. `constructorThrowsWhenPriceIsNotANumber`: linia `"1,Laptop,trei mii,2000,IT,5"` aruncă `NumberFormatException`.
4. `constructorThrowsWhenFieldsAreMissing`: linia `"1,Laptop"` aruncă tot o excepție. Află care: rulează o dată cu `assertThrows(Exception.class, ...)`, apoi pune tipul exact.

Done când: `Tests run: 5, Failures: 0`.

Exercițiul 2 · Primul mock: totalul coșului

În `CartTest`, `totalPriceMultipliesPriceByQuantityForEachItem`. Coșul are două produse:

Produs	id	preț	cantitate
laptop	1	3000	1
mouse	2	50	3

Produsele (`Product`) și rândurile din coș (`CartItem`) sunt `@Mock` , niciunul creat cu `new` . Le dai valorile cu `when(...).thenReturn(...)` , le pui în coș cu `addToCart` , apoi `totalPrice()` trebuie să fie `3150` .

Vezi în `Cart` ce metode cheamă `addToCart` și `totalPrice()` pe un `CartItem` și pe un `Product` : doar pe acelea pui `when` . Un `when` în plus face testul să pice cu `UnnecessaryStubbingException` .

Done când: `Tests run: 6, Failures: 0` .

Exercițiul 3 · verify: același produs de două ori

Tot în `CartTest` :

1. `addToCartIncrementsQuantityWhenProductIsAlreadyInCart` : două rânduri diferite (`laptopItem` , `secondLaptopItem`), dar cu același produs (id `1`). Primul are cantitatea `2` . După ce le adaugi pe amândouă, coșul are tot un singur rând, iar pe primul s-a apelat `setQuantity(3)` .
2. `removeFromCartLeavesCartEmptyWhenItWasTheOnlyItem` : adaugi un rând, îl scoți, coșul e gol.

`laptopItem` e un mock, deci `getQuantity()` întoarce mereu `2` , oricâte `setQuantity` primește. De aceea nu verifici cantitatea cu `assertEquals` , ci cu `verify` : ce i s-a cerut rândului.

Done când: `Tests run: 8, Failures: 0` .

Exercițiul 4 · @InjectMocks pe un serviciu

Clasa `ProductCommandServiceImplTest` în `test/app/products/service/` , cu `ProductRepository` și `ProductMapper` ca `@Mock` și serviciul `@InjectMocks` . Produsele cu care lucrezi sunt și ele `@Mock` .

1. `addProductSavesProductWhenNameIsNew` : repository-ul nu găsește niciun produs cu numele `Laptop` . După `addProduct` , repository-ul a primit `addProduct` cu exact acel produs.
2. `addProductThrowsWhenNameAlreadyExists` : repository-ul găsește deja un alt produs cu numele `Laptop` . `addProduct` aruncă `ProductAlreadyExistsException` , iar repository-ul nu primește niciun `addProduct` .

3. `removeProductThrowsWhenProductDoesNotExist`: produsul are id-ul `7`, iar repository-ul nu găsește nimic cu id-ul ăsta. `removeProduct` aruncă `ProductNotFoundException`, iar repository-ul nu primește niciun `removeProduct`.

`addProduct` aruncă o excepție verificată, deci testul de la punctul 1 are `throws Exception` în semnătură.

Verifică testele: în `ProductCommandServiceImpl.addProduct`, mută `productRepository.addProduct(product);` deasupra lui `if`. Testul 2 trebuie să pice cu `Never wanted here`. Pune linia înapoi.

Done când: `Tests run: 11, Failures: 0`.

Exercițiul 5 · ArgumentCaptor: ce s-a salvat

Tot în `ProductCommandServiceImplTest`. `createProduct` construiește singur produsul din cerere, cu `new`, deci nu ai un mock pe care să-l dai la `verify`.

1. `createProductSavesProductBuiltFromRequest`: cererea `("Laptop", 3000, 2000, "IT", 5)`. Prinzi cu un `ArgumentCaptor<Product>` produsul primit de `productRepository.addProduct` și verifici numele `Laptop`, prețul `3000` și stocul `5`.
2. `createProductThrowsWhenPriceIsNegative`: prețul `-1` aruncă `IllegalArgumentException`, iar repository-ul n-a fost atins deloc (`verifyNoInteractions`).

Done când: `Tests run: 13, Failures: 0`.

Exercițiul 6 · Fă serviciul testabil

`OrderCommandServiceImpl` își ia repository-ul cu `OrderRepositorySingleton.getINSTANCE()` în constructor, deci un test nu-l poate înlocui cu un mock.

1. În `src/`, adaugă un al doilea constructor, cu parametrii `OrderRepository` și `OrderMapper`. Constructorul fără argumente îl cheamă pe acesta cu `this(...)`, ca aplicația să meargă la fel. Uită-te la `ProductCommandServiceImpl`: acolo e deja făcut.
2. `OrderCommandServiceImplTest` în `test/app/orders/service/`, cu comenzile (`Order`) ca `@Mock`:
   - `addOrderSavesOrderWhenIdIsNew`: comanda cu id-ul `5` nu există, deci ajunge în repository;
   - `addOrderThrowsWhenOrderAlreadyExists`: există deja o comandă cu id-ul `5`, deci se aruncă `OrderAlreadyExistsException` și nu se salvează nimic;

- `deleteOrderRemovesOrderAndReturnsItsId`: `deleteOrder(new OrderDeleteRequest(5))` scoate comanda găsită, iar răspunsul are id-ul `5`;
- `deleteOrderThrowsWhenOrderDoesNotExist`: pentru id-ul `9` se aruncă `OrderNotFoundException` și nu se scoate nimic.

Încearcă întâi testele fără pasul 1: pică cu `Wanted but not invoked` și `UnnecessaryStubbingException`, fiindcă serviciul lucrează cu repository-ul adevărat, nu cu mock-ul tău.

Done când: `Tests run: 17, Failures: 0`, iar aplicația pornește din `Main` ca înainte.

Exercițiul 7 · Utilizatori și permisiuni

Fă testabil `UserCommandServiceImpl` la fel ca la exercițiul 6 (are 3 dependențe). `UserCommandServiceImplTest` în `test/app/users/service/`, cu utilizatorii (`User`) ca `@Mock`. `User` e o clasă abstractă: Mockito face mock și pe ea.

1. `registerUserReturnsTheSavedUser`: nu există utilizatorul `ana` / `1234`. `registerUser` întoarce exact obiectul dat de `saveUser` (`assertSame`).
2. `registerUserThrowsWhenUserAlreadyExists`: `ana` / `1234` există deja, deci se aruncă `UserAlreadyExistException` și nu se salvează nimic.
3. `grantExtraPermissionAddsPermissionAndUpdatesUser`: utilizatorul `1` are rolul `Role.CUSTOMER` și primește `VIEW_USERS`. Mesajul e `Permisiunea extra a fost adaugata`, iar utilizatorul ajunge la `updateUser`.
4. `grantExtraPermissionDoesNothingWhenRoleAlreadyHasIt`: `VIEW_PRODUCTS` vine deja din rolul `CUSTOMER`. Mesajul e `Permisiunea exista deja prin rolul utilizatorului`, iar `addExtraPermission` și `updateUser` nu se apelează.
5. `grantExtraPermissionThrowsWhenUserDoesNotExist`: utilizatorul `9` nu există, deci se aruncă `UserNotFoundException`.

`Role` e un `enum`: nu-l faci mock, îi dai valoarea reală `Role.CUSTOMER`.

Done când: `Tests run: 22, Failures: 0`.

Provocări

Fără indicii, doar ce trebuie să fie adevărat la final. Fiecare serviciu îl faci întâi testabil, ca la exercițiul 6.

Exercițiul 8 · OrderQueryServiceImpl

`OrderQueryServiceImplTest`, 4 teste:

- clientul `2` are două comenzi, iar `findOrdersByCustomerId(2)` întoarce exact acele două comenzi;
- clientul `9` n-are comenzi, deci se aruncă `CustomerNotFoundException`;
- `checkOrderId(3)` e `true` când lista are o comandă cu id-ul `3`;
- `checkOrderId(8)` e `false` pe aceeași listă.

Done când: `Tests run: 26, Failures: 0`.

Exercițiul 9 · updateProduct

În `ProductCommandServiceImplTest`, 4 teste noi pe `updateProduct(ProductUpdateRequest)`:

- produsul nu există, deci se aruncă `ProductNotFoundException` și nu se actualizează nimic;
- numele nou e al altui produs, deci se aruncă `ProductAlreadyExistsException`, iar produsul nu primește `setName`;
- același produs își păstrează numele și primește prețul nou, deci ajunge la `updateProduct`;
- stocul negativ aruncă `IllegalArgumentException`, iar nimic nu se actualizează.

Done când: `Tests run: 30, Failures: 0`.

Exercițiul 10 · OrderDetailCommandServiceImpl

`OrderDetailCommandServiceImplTest`, 4 teste:

- `createOrderDetail` pe cererea `(3, 1, 3000f, 2)` salvează un detaliu cu exact aceste valori;
- un detaliu cu un id care există deja nu se salvează (`OrderDetailAlreadyExistsException`);
- `deleteOrderDetail` pe id-ul `4` scoate detaliul și întoarce id-ul `4`;
- `deleteOrderDetail` pe un id inexistent aruncă `OrderDetailNotFoundException`.

Done când: `Tests run: 34, Failures: 0`, iar aplicația pornește din `Main` ca înainte.